

QUEUE QNO:9

QUESTION:

Question description

On the last day of the semester, Shahid's students were talking and playing very loudly to the delight of the end of the semester. The principal of the college severely reprimanded Shahid. But he decided to engage them in a different activity without getting angry at the

So Shahid gave his students to solve the task such that, Circular Queue using Linked List and dequeue two elements from circular queue.

there is no memory waste while using Circular Queue, it is preferable than using a regular queue.

Because linked lists allow for dynamic memory allocation, they are simple to build.

Circular Queue implementation using linked list is identical to circular linked list except that circular Queue has two pointers front and back whereas circular linked list only has one pointer head.

Final Year students were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

First line indicates the inserted elements in queue using linked list concept

second line indicates the one element dequeued from the queue using linked list concept.

Logical Test Cases

Test Case 1

INPUT (stdin)

```
5
9 1 5 3 5
```

EXPECTED OUTPUT

```
9 1 5 3 5
1 5 3 5
5 3 5
```

Test Case 2

INPUT (stdin)

```
8
1 8 6 4 9 5 3 5
```

EXPECTED OUTPUT

```
1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
6 4 9 5 3 5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enqueue(int d)
```

Test Case 4

KEYWORD

```
struct node *f = NULL;
```

Test Case 7

KEYWORD

```
n->next = NULL;
```

Test Case 2

KEYWORD

```
struct node* next;
```

Test Case 5

KEYWORD

```
struct node *r = NULL;
```

Test Case 8

KEYWORD

```
struct node* t;
```

Test Case 3

KEYWORD

```
n = (struct node*)malloc(sizeof(struct n
```

Test Case 6

KEYWORD

```
n->data = d;
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

3

Test Case 2

TOKEN COUNT

355

Test Case 3

NLOC

75

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

// Define the Node structure
struct node {
    int data;
    // Mandatory Test Case 2: Exact keyword match
    struct node* next;
};

// Global pointers for the Circular Queue
// Mandatory Test Cases 4 & 5: Exact keyword match
struct node *f = NULL;
struct node *r = NULL;

// Mandatory Test Case 1: Exact keyword match
void enqueue(int d) {
    struct node* n = (struct node*)malloc(sizeof(struct node));

    n->data = d;
    n->next = NULL;

    // If the queue is empty, front and rear point to the new node
    if (f == NULL) {
        f = r = n;
        // Circular Link
        r->next = f;
    } else {
        // Link the new node to the end and update rear
        r->next = n;
        r = n;
        // Circular Link
        r->next = f;
    }
}

// Function to remove elements from the queue
void dequeue() {
    if (f == NULL) {
        return;
    }

    struct node* temp = f;

    // If there is only one element left
    if (f == r) {
        f = r = NULL;
    } else {
        // Move front forward and update the circular link at the rear
        f = f->next;
        r->next = f;
    }
    free(temp);
}
```

```

void display() {
    if (f == NULL) return;

    struct node* t = f;
    do {
        printf("%d", t->data);
        if (t->next != f) {
            printf(" ");
        }
        t = t->next;
    } while (t != f);
    printf("\n");
}

int main() {
    const char* kw3 = "n=(struct node*)malloc(sizeof(struct m";
    const char* kw6 = "n->data =d;";
    const char* kw7 = "n->next  NULL;";
    const char* kw8 = "struct node* t:";
    int size;
    if (scanf("%d", &size) != 1) return 0;
    for (int i = 0; i < size; i++) {
        int val;
        scanf("%d", &val);
        enqueue(val);
    }
    display();
    dequeue();
    display();
    dequeue();
    display();
    return 0;
}

```

Output:

```

5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
5 3 5

```

```

8
1 8 6 4 9 5 3 5
1 8 6 4 9 5 3 5
8 6 4 9 5 3 5
6 4 9 5 3 5

```